System Architecture Design For Automated Container Inspection In Ports

Aththariq Lisan Quran Daulah Sentono
Institut Teknologi Bandung
Bandung, Indonesia
18222013@std.stei.itb.ac.id

I Gusti Bagus Baskara Nugraha
Institut Teknologi Bandung
Bandung, Indonesia
baskara@itb.ac.id

***Abstract*—Port container inspection produces identity observations, visual evidence, operator decisions, and reports, but these artifacts are often fragmented across camera feeds, checklist records, image files, and delayed reporting channels. This paper presents Cargovision, a human-in-the-loop software architecture for computer-vision-based port container inspection. The contribution is not a detector, deployment, or governance benchmark; it is an architecture artifact that defines layer responsibilities, component contracts, and a container-centered inspection data contract. The method follows design science research and translates inspection needs into a four-layer architecture: Presentation Layer, Application Layer, Data Layer, and Edge Layer. The architecture binds OCR identity, scan evidence, operator review, structured persistence, and report preparation to a central Container record and its inspection evidence; separates REST/API writes from WebSocket live video; stores visual artifacts as object references outside transactional state; and prepares adapter boundaries for future TOS, PCS, Inaportnet, INSW, and API Ditjen Hubla integration. Scenario-based evaluation shows that the Cargovision prototype realizes the five prototype-scope decisions, while the external-adapter boundary remains design intent and production readiness still requires larger field validation, external-system integration, and longer reliability testing.**

***Index Terms*—software architecture, container inspection, human-in-the-loop, edge computing, inspection data contract, maritime logistics.**

I. Introduction

Maritime container terminals depend on inspection records that are accurate, traceable, and available soon after a truck passes the gate. One passage can involve visible container marks, exterior condition, camera images, operator judgment, and later reporting. When these elements are captured through separate manual or semi-digital channels, the inspection history becomes hard to reconstruct. The architectural problem is therefore not only whether a model can read a number or mark a defect; it is whether identity, evidence, review, persistence, and reports can be bound into one durable container record. Cargovision frames this problem as process standardization, information integration, and operational reliability.

Manual inspection guidance such as container checklist practice and equipment-inspection criteria provides procedural structure, but it does not define how software should preserve evidence, coordinate uncertain machine output, or expose reviewable records across inspection components [1], [2]. Non-intrusive inspection systems improve screening capability, yet they are commonly discussed as infrastructure or imaging systems rather than as software architectures for container-owned inspection history [3], [4]. Computer-vision research contributes optical character recognition (OCR) and damage-detection techniques [5]–[7], but model-level outputs remain insufficient without an application boundary that handles identity correction, duplicate suppression, audit trails, and human authority.

This paper addresses that gap through Cargovision's architecture. The proposed architecture treats Container as the central product state, not raw frames, isolated tracking events, or model detections. The edge layer observes camera streams and prepares OCR and defect evidence. The application layer validates machine-to-machine requests, normalizes container identity, persists scan summaries, and guards canonical writes. The presentation layer supports live monitoring, operator review, history, and report access. Human-in-the-loop review remains explicit because outdoor OCR and defect detection can fail under blur, partial views, lighting variation, and occlusion; accountable inspection requires a place for authorized personnel to confirm, correct, reject, hold, or escalate automated evidence [8].

The objective is to define and evaluate a container-centered inspection architecture. The contributions are: (1) a four-layer architecture decision set for port inspection, covering Presentation Layer, Application Layer, Data Layer, Edge Layer, local edge capability, Container as integration anchor, structured-data/live-video separation, artifact/state separation, and adapter readiness for future TOS, PCS, Inaportnet, INSW, and API Ditjen Hubla integration; (2) a container-centered information contract and UML activity view that separate machine evidence candidates from durable inspection state; and (3) a scenario-backed evaluation that checks whether the architecture decisions are observable in the prototype artifact. The scope is intentionally narrower than the full collaborative platform: detector development, deployment operations, governance scoring, and production procedures are adjacent concerns.

II. Related Work

Inspection standardization is the first relevant stream. Container security and equipment-inspection guidance defines

what officers should check, how visible anomalies are categorized, and why traceability matters [1], [2]. These materials are useful for requirements but do not prescribe an application architecture for evidence capture, identity stabilization, review workflow, or persistence. They leave open the software question of how observations become a consistent, auditable, container-owned record.

The second stream is inspection automation and imaging infrastructure. Non-intrusive inspection equipment and X-ray systems strengthen security screening and can reduce purely manual inspection load [3], [4]. However, screening infrastructure and application-state architecture solve different problems. A port can have imaging capability and still lack a software contract that joins identity, evidence, operator decisions, report history, and integration boundaries. Cargovision therefore uses inspection-automation research as context, but positions its contribution at the information-system architecture layer.

The third stream is computer vision for container identification and defect detection. Recent OCR work improves container-code recognition under environmental interference [5], [6]. Damage-detection research reports progress in learning defect classes from image datasets [7]. These works support the feasibility of machine-generated evidence, but they do not remove the need for a human-in-the-loop architecture. A detector-centered system can still create ghost records, duplicate writes, or unreviewed operational decisions if the architecture promotes every raw output into product state.

The fourth stream is smart-port digitalization and software architecture. Smart-port studies emphasize process improvement, digital integration, and terminal-wide transformation [9], [10]. Software architecture literature emphasizes modular responsibilities, bounded communication, and quality-attribute tradeoffs [11], [12]. Edge-computing research further motivates local processing close to sensors when latency, bandwidth, or operational continuity matter [13]. The architectural claim is not that layering, aggregate ownership, or REST/WebSocket separation are new patterns; it is that these patterns are composed around uncertain computer-vision evidence, human authority, and container identity stabilization in a port-inspection workflow. Cargovision combines these directions by assigning camera-facing inference to the edge, preserving canonical writes through the API, and keeping the operator review workflow explicit.

III. Method

The work follows design science research methodology: identify the operational problem, define artifact objectives, construct the artifact, demonstrate it, and evaluate it against the design purpose [14], [15]. The artifact is the software architecture of integrated container inspection. Its unit of analysis is not an isolated model, endpoint, or screen, but the responsibility structure that moves from observed evidence to a reviewable container record.

The design analysis produced five architecture concerns: coordinate inspection process state, persist structured container data, store unstructured visual evidence, exchange information through explicit contracts, and present inspection results to operators. These concerns map to maintainability through modular boundaries, interoperability through stable contracts, auditability through activity and evidence history, responsiveness through local edge processing, and reliability through separation between live monitoring and durable persistence [11], [16].

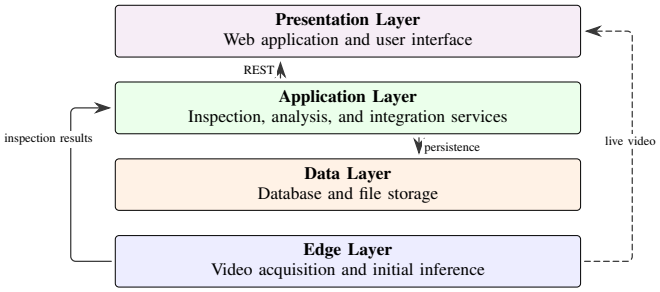

Fig. 1. Condensed rendering of the layered architecture. The figure preserves the four architecture layers and communication paths while implementation-specific components are discussed separately in text and tables.

Five solution styles were compared at the architecture level: process-and-data standardization, orchestrated integration, domain-based modularity, event-driven workflow, and integrated data-governance architecture. The comparison used the five concerns above as selection criteria. Process-and-data standardization clarified vocabulary but did not define executable software boundaries. Orchestrated integration and event-driven workflow introduced central coordination or broker dependencies beyond the prototype's needs. Integrated data-governance architecture fit organization-level ownership better than local inspection-system structure. Domain-based modular architecture was selected because it covers the concerns with explicit software boundaries while keeping the prototype independent of central orchestration, broker infrastructure, or organization-level governance procedures. In Cargovision, this decision is realized as four layers: Presentation Layer, Application Layer, Data Layer, and Edge Layer.

Fig. 1 is a condensed rendering of the layered architecture. It preserves the four-layer structure and the same communication meanings: REST access between presentation and application, persistence from application to data, inspection-result submission from edge to application, and direct live-video streaming from edge to presentation. More detailed implementation names such as `/api/edge`, object storage, and external adapters are discussed in the text and traceability tables so the figure remains an architecture view rather than a component inventory.

The inspection contract is represented in two complementary views. Fig. 2 shows the activity lifecycle from camera evidence to reviewed container history. OCR and defect outputs remain evidence candidates until validation, stability, or operator correction promotes a canonical Container identity [17]. Live annotated frames support monitoring, but the durable record stores structured inspection state and media references, not binary video.

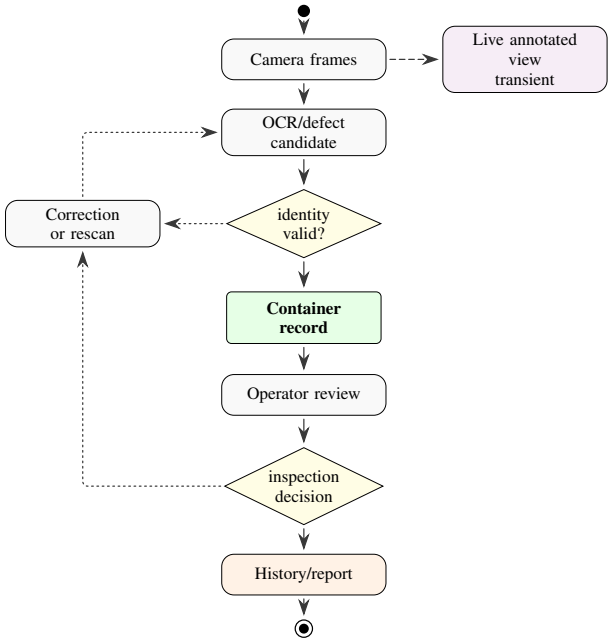

Fig. 2. UML-style activity view of the inspection evidence lifecycle. Automated OCR and defect outputs remain candidates until validated or corrected; reviewed outcomes attach to Container history while live video remains transient.

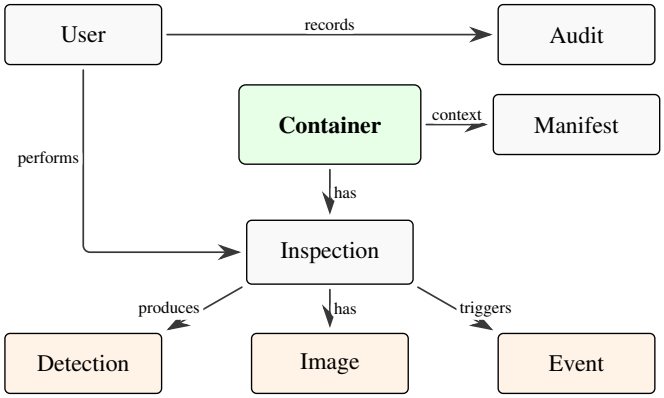

Fig. 3. Condensed conceptual data contract for Cargovision inspection records. Container is the central entity, with Manifest and related inspection entities shown around it.

The structured edge-write contract is intentionally narrow: it carries source context, identity evidence, scan evidence, media references, and idempotent review controls. This keeps frame-level uncertainty and high-volume video out of the canonical product state while preserving enough evidence for audit and operator review.

Fig. 3 complements the lifecycle view by showing the conceptual data contract. The model places Container at the center, retains Manifest as supporting context, and relates inspection records, users, audit history, detection evidence, images, and events to that central record.

The realization evidence uses a camera-facing edge service, inspection API, operational data store, object store, and web surface. The web application consumes two channels: structured inspection data through the API and live frames directly from the edge service. The credential-protected `/api/edge` family represents the structured machine-to-machine write contract, while the WebSocket channel represents transient monitoring. The API is deliberately not a video relay. Specific frameworks, database products, and cloud vendors are implementation evidence only, not the contribution.

IV. Evaluation, Results, and Discussion

The evaluation checks whether the architecture decisions in Table I are observable in the Cargovision artifact. It uses architecture-conformance and design-traceability validation rather than a production field trial [11]. The scenarios cover layer communication, edge evidence preparation, container-centered OCR and scan attachment, visual-artifact storage, live-video separation, and adapter readiness.

The procedure was a desk-and-prototype walkthrough by the first author against local repositories and recorded checks. Evidence included OCR-created container records, authenticated `/api/edge` defect-scan writes, container-number propagation, live WebSocket-style display, container detail/report rendering, manual review, and invalid-credential rejection. A scenario was marked satisfied only when both the implementation surface and an observed prototype check supported the architecture criterion.

Table II summarizes the results. Five artifact-backed architecture decisions were satisfied: layer separation, canonical edge-to-API writes, container-centered scan attachment, artifact/state separation, and transient live video. The external-adapter item remains design intent because no external integration has been completed.

The evaluation also checks two integration risks. First, the candidate-to-canonical rule prevents raw OCR previews from directly becoming durable product state, although unstable OCR under blur, occlusion, or lighting variation still needs larger field testing. Second, live video stays on a transient edge-to-web path while the API carries structured inspection state. These are architecture-level observations, not claims of comprehensive field performance.

The main validity threats are evaluator bias, prototype-only scope, and limited external integration. Here, "satisfied" means that a decision is present and supported by prototype checks, not that a quality attribute has been independently measured. Model performance, deployment reliability, governance procedures, and stakeholder ownership remain adjacent work beyond this architecture evaluation.

V. Conclusion

This paper presented Cargovision as a container-centered software architecture for human-in-the-loop port container inspection. The design uses domain-based modularity to separate Presentation Layer, Application Layer, Data Layer, and Edge Layer responsibilities: operator-facing review, API-controlled persistence, database and object-storage concerns, and camera-facing evidence acquisition. Its central contract is that automated OCR and defect outputs are evidence candidates,

TABLE I
ARCHITECTURE DECISION AND INFORMATION-CONTRACT TRACEABILITY

Architecture Need	Architecture Decision	Realization or Evaluation Evidence	Boundary
Modular evolution	Four named layers: Presentation, Application, Data, Edge	Each layer owns a distinct responsibility and communicates through service contracts	Not a framework comparison
Responsive acquisition	Local edge capability near camera source	Edge prepares OCR/defect evidence and live monitoring before canonical API persistence	Not a latency benchmark
Inspection context	Container as integration anchor	OCR identity, scan summaries, review state, manifest context, history, and report metadata bind to one container-centered conceptual data model	Not a model-accuracy claim
Mixed data loads	Structured data and live video separated	Credential-protected `/api/edge` and database paths carry validated inspection state; WebSocket channel carries transient annotated frames	Not a deployment claim
Evidence management	Visual artifacts separated from transactional state	Container records store metadata and object references; image/report assets remain outside the operational document	Retention outside scope
External heterogeneity	Adapter-ready integration boundary	Core inspection logic is isolated from future target-system adapters	No completed external integration claimed

TABLE II
SCENARIO-BACKED ARCHITECTURE EVALUATION RESULTS

Scenario Focus	Observed Architecture Evidence	Boundary
Layered/service separation	Presentation Layer, Application Layer, Data Layer, and Edge Layer responsibilities remain separated by service contracts across web review, API writes, persistence, storage, and camera-facing evidence preparation.	Internal scope
Local edge capability	Edge path submits OCR/defect evidence and provides live monitoring before canonical API persistence.	No latency benchmark
Container integration anchor	OCR formation, container-number propagation, scan summaries, review state, history, and report paths attach to the same container-centered inspection record.	Accuracy outside scope
Structured-data/live-video separation	Structured inspection records remain on the authenticated `/api/edge`/database path while annotated frames remain transient WebSocket-style edge-to-web data.	Field network pending
Visual-artifact separation	Detail and report checks render persisted container state with evidence references; heavy media remains outside the main record.	Retention external
External adapter boundary	Core API contracts are intended to isolate planned target-system adapters from the core inspection record; adapter extensibility has not yet been exercised.	Design intent only

while the durable product state remains a reviewable container-centered inspection record.

Scenario-based evaluation shows that five prototype-backed architecture decisions can be realized across the Cargovision prototype's edge, API, web, database, and storage components, while the external adapter boundary remains a forward-looking extension. The result supports architecture feasibility for prototype-scope inspection automation: structured evidence can be written through controlled endpoints, visual evidence can be preserved separately, live video can remain transient, and operators can review persisted container records. Future work should extend the evidence with larger field OCR and defect evaluations, write-path load tests, longer reliability runs, formal practitioner validation, and adapter-based integration with the target external systems.

REFERENCES

[1] U.S. Customs and Border Protection, "7-point container inspection checklist," https://www.cbp.gov/document/guides/ctpat-7-point-container-inspection-checklist, 2022, accessed: 2025-10-22.

[2] Institute of International Container Lessors, *Guide for Container Equipment Inspection, 6th Edition*. Washington, DC, USA: IICL, 2016.

[3] World Customs Organization, "Guidelines for the procurement and deployment of scanning and non-intrusive inspection equipment," https://www.wcoomd.org/-/media/wco/public/global/pdf/topics/facilitation/instruments-and-tools/tools/safe-package/nii-guidelines-2020/nii-guidelines-en_sep-2020.pdf?la=en, 2020, accessed: 2025-10-22.

[4] C. Lim, J. Lee, Y. Choi, J. W. Park, and H. K. Kim, "Advanced container inspection system based on dual-angle x-ray imaging method," *Journal of Instrumentation*, vol. 16, no. 8, 2021.

[5] Z. Zhang, Y. Ding, R. Li, and K. Chen, "Enhancing OCR with line segmentation mask for container text recognition in container terminal," *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108667, 2024.

[6] M. Yu, S. Zhu, B. Lu, Q. Chen, and T. Wang, "A two-stage automatic container code recognition method considering environmental interference," *Applied Sciences*, vol. 14, no. 11, p. 4779, 2024.

[7] T. N. T. Phuong, G. S. Cho, and I. Chatterjee, "Automating container damage detection with the YOLO-NAS deep learning model," *Science Progress*, vol. 108, no. 1, 2025.

[8] K. Lazaros, A. G. Vrahatis, and S. Kotsiantis, "Human-in-the-loop artificial intelligence: A systematic review of concepts, methods, and applications," *Entropy*, vol. 28, no. 4, p. 377, 2026.

[9] M. Heikkila, J. Saarni, and A. Saurama, "Innovation in smart ports: Future directions of digitalization in container ports," *Journal of Marine Science and Engineering*, vol. 10, no. 12, p. 1925, 2022.

[10] J. Basulo-Ribeiro, C. Pimentel, and L. Teixeira, "Digital transformation in maritime ports: Defining smart gates through process improvement in a portuguese container terminal," *Future Internet*, vol. 16, no. 10, p. 350, 2024.

[11] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2022.

[12] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[13] W. Shi and S. Dustdar, “The promise of edge computing,” *Computer*, vol. 49, no. 5, pp. 78–81, 2016.
[14] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
[15] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
[16] ISO/IEC, “ISO/IEC 25010:2023: Systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – product quality model,” https://www.iso.org/standard/78176.html, 2023, international standard.
[17] International Organization for Standardization, “ISO 6346: Freight containers – coding, identification and marking,” https://www.iso.org/standard/83558.html, 2022, international standard.